Rapport final de projet - Quizora

Application de création et de passation de quiz en Python

Projet	Quizora
Module	PROJ531 - Gestion de projet informatique
Établissement	Université Savoie Mont Blanc
Équipe	Bilal Draoui (Product Owner), Sabrine Kawiche, Chadi Kharchoufa, Gabrien Vaz
Période	Du 13 octobre au 9 novembre 2026 (5 séances, 4 sprints)
Dépôt GitHub	https://github.com/KaSabrin/PROJ531-Quiz-Python
Tableau Trello	https://trello.com/b/zFAmvu0D/proj531-quiz-python

---

Introduction

Ce rapport présente le bilan du projet **Quizora**, une application Python de création et de passation de quiz réalisée par une équipe de quatre étudiants dans le cadre du module PROJ531. Le projet avait un double objectif : livrer une application fonctionnelle et mettre en pratique les méthodes de gestion de projet (organisation en équipe, planification, suivi de l'avancement, travail collaboratif avec Git).

Le rapport suit les critères d'évaluation du sujet : objectifs du projet (A), gestion du projet (B), autonomie de l'équipe et gestion des difficultés (C), réalisation technique (D), bilan final (E). Il fait référence aux livrables déposés dans le dossier `docs/` du dépôt.

A. Objectifs du projet

A.1 Compréhension du sujet

Le sujet demande de réaliser une application Python permettant de créer des quiz, c'est-à-dire des séries de questions auxquelles des utilisateurs autorisés peuvent répondre. L'application doit afficher le score final et les bonnes réponses.

Nous avons identifié les exigences suivantes :

Type	Exigence du sujet	Interprétation de l'équipe
Minimum	Création de compte	Inscription et connexion sécurisées, avec deux rôles : joueur et administrateur
Minimum	Historique des scores individuels	Chaque passage de quiz est enregistré et consultable par le joueur
Minimum	Création de quiz par un administrateur, directement ou depuis un fichier texte	Formulaire de création interactif et import d'un fichier `.txt` au format défini par l'équipe

Type	Exigence du sujet	Interprétation de l'équipe
Minimum	Affichage du score final et des bonnes réponses	Correction affichée après chaque question et récapitulatif des erreurs en fin de quiz
Optionnel	Interface graphique	Réalisée avec Tkinter
Optionnel	Minuterie, export JSON	Non réalisées (voir partie E)

A.2 Objectifs de gestion de projet

Le sujet impose également une organisation de type Scrum : un Product Owner, des tâches de deux jours maximum, un suivi sur Trello, un dépôt GitHub où chaque membre est collaborateur, un compte rendu à chaque fin de séance et une version sans bug disponible à la fin de chaque séance. Les livrables attendus sont un planning (Gantt, PBS, WBS et planning des sprints), un modèle conceptuel de données, le code source avec son README et les comptes rendus, et le présent rapport.

B. Gestion du projet

B.1 Organisation de l'équipe

Membre	Rôle	Principales responsabilités
Bilal Draoui	Product Owner et développeur	Priorisation du backlog, validation des livrables, communication avec l'enseignant, README, interface graphique, comptes rendus
Sabrine Kawiche	Développeuse	Modélisation du MCD, gestion du dépôt GitHub, création manuelle de quiz, sauvegarde des scores
Chadi Kharchoufa	Développeur	WBS et Gantt, sécurité des mots de passe, import de quiz depuis un fichier texte, service des résultats et ses tests
Gabrien Vaz	Développeur	Tableau Trello, stockage JSON, passation de quiz côté joueur, historique des scores

Le dépôt GitHub a été créé en début de projet et chaque membre y a été ajouté comme collaborateur avec les droits d'écriture.

B.2 Ressources du projet

- **Humaines :** quatre étudiants, pendant les séances encadrées et du travail personnel entre les séances.
- **Logicielles :** Python 3.10 et plus, uniquement avec la bibliothèque standard (aucune dépendance à installer) ; Visual Studio Code ; Git et GitHub ; Trello ; GanttProject ; Looping pour le MCD.
- **Matérielles :** les ordinateurs personnels des membres. Aucun serveur ni base de données n'est nécessaire, les données étant stockées dans des fichiers JSON locaux.

B.3 Modèle de processus et planification

Nous avons suivi une démarche **agile inspirée de Scrum** : le travail a été découpé en quatre sprints successifs, chacun livrant une partie fonctionnelle de l'application, avec des tâches ne dépassant pas deux

jours.

Sprint	Dates prévues	Contenu	Livrables
Sprint 1 - Cadrage et conception	12/10 - 15/10	Dépôt GitHub, README, MCD, WBS, Gantt, Trello	`mcd_quizora.png`, `mcd_documentation.md`, `wbs_projet_quiz_python.png`, `Gantt_projet_quiz.gan`
Sprint 2 - Authentification et stockage	19/10 - 22/10	Architecture du code, comptes utilisateurs, hachage des mots de passe, stockage JSON, menu console	`src/models/`, `src/services/auth_service.py`, `src/security.py`, `storage_service.py`, 15 tests
Sprint 3 - Cœur du jeu	26/10 - 30/10	Création manuelle de quiz, import depuis un fichier texte, passation de quiz, page de connexion graphique	`quiz_service.py`, `import_service.py`, `src/gui.py`
Sprint 4 - Scores, finitions et soutenance	02/11 - 06/11	Historique des scores, chasse aux bugs, interface graphique complète, rapport, soutenance	`resultat_service.py`, quiz d'exemple, 35 tests, présent rapport

Les livrables de planification sont disponibles dans `docs/` :

- **WBS** (`wbs_projet_quiz_python.png`) : décomposition des travaux en quatre axes, un par sprint ;
- **PBS** (`PBS_Quizora.png`) : décomposition du produit en ses composants ;
- **Gantt** (`Gantt_projet_quiz.gan` et `.png`) : planification des tâches dans le temps, avec des tâches de deux jours au plus ;
- **MCD** (`mcd_quizora.png` et `mcd_documentation.md`) : modèle Entité-Association des données.

B.4 Suivi de l'avancement

- **Trello :** un tableau Kanban (Backlog, À faire, En cours, À vérifier, Terminé) a servi à définir les user stories (US01 à US07, puis US11 pour l'interface graphique), à les découper en tâches, à les attribuer et à suivre leur avancement. Les captures du tableau aux sprints 2, 3 et 4 sont jointes aux comptes rendus.
- **Comptes rendus de séance :** un compte rendu a été déposé sur GitHub à la fin de chacune des cinq séances (`docs/comptes-rendus/`). Il indique les tâches réalisées, leur responsable, le temps estimé et le temps réellement passé, les décisions prises et les difficultés rencontrées.
- **GitHub :** l'historique du dépôt compte plus de **80 commits** des quatre membres entre le 13 octobre et le 12 novembre 2026. Il permet de retracer qui a réalisé chaque modification.

B.5 Démarche d'amélioration continue

À chaque séance, l'équipe a fait le point sur l'avancement et a ajusté ses choix. Les comptes rendus gardent la trace de ces décisions, par exemple :

- **Séance 2 :** le champ `mdp` prévu dans le MCD a été remplacé par `mdp_hash` pour ne jamais stocker de mot de passe en clair ; l'encodage UTF-8 a été imposé pour éviter les problèmes d'accents sous Windows ;
- **Séance 3 :** les emojis ont été retirés de l'ensemble du projet (console, services, documentation) ;
- **Séance 4 :** une relecture complète du code a permis de corriger sept anomalies et de supprimer du code inutilisé ;
- **Séance 5 :** retour d'expérience sur notre utilisation de Git, qui alimente les pistes d'amélioration de la partie E.

C. Autonomie de l'équipe et gestion des difficultés

C.1 Auto-formation

Plusieurs notions ont dû être apprises pendant le projet :

- **Git et GitHub en équipe :** récupération du travail des autres (`git pull`), résolution de conflits, branches et Pull Requests, fichier `.gitignore` ;
- **Sécurité :** hachage des mots de passe avec sel (`hashlib`, `secrets`) et comparaison en temps constant (`hmac.compare_digest`) ;
- **Tkinter**, pour réaliser l'interface graphique ;
- **Tests unitaires** avec le module `unittest`, en utilisant des dossiers temporaires pour ne jamais modifier les vraies données ;
- **Markdown**, pour le README et les comptes rendus.

C.2 Difficultés rencontrées et solutions apportées

Difficulté	Solution
Périmètre du MCD (séance 1) : fallait-il enregistrer chaque réponse du joueur ou seulement son score ?	Conformément au minimum demandé (historique des scores), l'entité `Résultat` a été liée à `Utilisateur` et à `Quiz`, sans enregistrer chaque réponse.
Découpage en tâches de deux jours au plus (séance 1)	Découpage en quatre sprints progressifs et en tâches courtes dans le WBS et le Gantt.
Stockage des mots de passe (séance 2)	Hachage SHA-256 avec un sel aléatoire de 16 octets propre à chaque utilisateur.
Conflit Git sur `src/cli.py` (séance 4) : deux membres avaient modifié la même ligne du menu joueur.	Conflit résolu en conservant la version de la coéquipière ; la relecture a aussi révélé qu'une méthode appelée n'existait pas encore, ce qui faisait planter l'application. La méthode a été ajoutée.
Anomalies découvertes en fin de projet (séance 4) : boucle infinie à la création d'un quiz, plantage à l'import d'un fichier non texte, barème `[1 pt]` mal lu, tests d'import jamais exécutés	Corrections et ajout de tests de non-régression ; la suite passe de 19 à 35 tests.

Difficulté	Solution
Commits non reliés aux comptes GitHub : l'adresse e-mail utilisée dans Git n'était pas enregistrée sur le compte GitHub.	Ajout et vérification de l'adresse dans les paramètres GitHub ; les commits ont alors été rattachés automatiquement au bon compte.
Dépassement du planning : le Gantt prévoyait une fin le 06/11.	Le rapport et la répétition de la soutenance ont été finalisés lors d'une cinquième séance (09/11).

C.3 Prise d'initiatives et cohésion de l'équipe

Au-delà du minimum demandé, l'équipe a pris l'initiative de réaliser une interface graphique complète, de rédiger un dossier d'architecture technique, un glossaire et un PBS, et d'ajouter des quiz de démonstration qui permettent de tester l'application immédiatement après l'avoir téléchargée. La répartition des tâches par modules (interface, services, modèles) a permis à chacun de travailler sur sa partie en limitant les modifications simultanées des mêmes fichiers.

D. Réalisation technique

D.1 Architecture logicielle

L'application suit une **architecture en couches**, qui sépare l'interface, la logique métier et le stockage. Le schéma détaillé est disponible dans `docs/architecture_technique_quizora.pdf`.

Couche	Fichiers	Rôle
Présentation	`main.py`, `gui_main.py`, `src/cli.py`, `src/gui.py`	Deux interfaces au choix : console ou fenêtre graphique Tkinter
Services métier	`src/services/` : `auth_service.py`, `quiz_service.py`, `import_service.py`, `resultat_service.py`, `storage_service.py`	Inscription et connexion, création et import de quiz, scores et historique, lecture et écriture des fichiers
Sécurité	`src/security.py`	Hachage et vérification des mots de passe
Modèles	`src/models/` : `User`, `Quiz`, `Question`, `Reponse`, `Resultat`	Entités du MCD, avec conversion vers et depuis JSON (`to_dict` / `from_dict`)
Persistance	`data/users.json`, `data/quizzes.json`, `data/results.json`	Données locales, créées automatiquement au premier lancement

Grâce à cette séparation, la console et l'interface graphique utilisent **exactement les mêmes services** : l'interface graphique a pu être ajoutée sans modifier la logique métier, et cette logique est testée indépendamment de l'interface. Le code source compte environ 2 750 lignes, et les tests environ 440 lignes.

D.2 Modèle de données

Le MCD comporte cinq entités : **Utilisateur**, **Quiz**, **Question**, **Réponse** et **Résultat**. Un utilisateur crée des quiz (0,n) et obtient des résultats (0,n) ; un quiz contient au moins une question (1,n) ; une question

propose au moins deux réponses (2,n) ; un résultat concerne un utilisateur et un quiz. Le détail des attributs et des cardinalités est dans `docs/mcd_documentation.md`.

D.3 Choix techniques et justifications

Choix	Solutions envisagées	Justification
Stockage en fichiers JSON	Base de données SQLite, fichiers CSV	Format lisible, natif en Python, sans installation ; adapté au volume de données d'un projet pédagogique.
Bibliothèque standard uniquement	Frameworks d'interface ou de sécurité externes	Installation immédiate (un simple `git clone`), aucun problème de version entre les postes des membres.
SHA-256 avec sel aléatoire et comparaison en temps constant	Stockage en clair, hachage sans sel	Les mots de passe ne sont jamais stockés en clair ; le sel empêche l'utilisation de tables de hachage précalculées.
Import au format texte simple	JSON, CSV	Un enseignant peut écrire un quiz dans un simple éditeur de texte. Format : `TITRE:`, `DESCRIPTION:`, `QUESTION: ... [2 pts]`, `*` pour la bonne réponse, `-` pour les mauvaises.
Deux interfaces (console et Tkinter)	Console seule	La console a permis de livrer rapidement les fonctionnalités ; Tkinter, inclus dans Python, offre une interface plus accessible sans dépendance supplémentaire.
Dossier `data/` exclu de Git	Versionner les données	Ne pas publier les comptes des utilisateurs, et éviter les conflits entre les données de test de chaque membre.

D.4 Validation

La validation repose sur **35 tests unitaires automatisés**, exécutés avec `python -m unittest discover tests` avant chaque envoi sur GitHub :

Fichier	Tests	Ce qui est vérifié
`test_security.py`	4	Format du hachage, unicité grâce au sel, vérification correcte ou refusée
`test_auth.py`	9	Inscription valide, refus des doublons, des e-mails invalides et des mots de passe faibles, connexion et déconnexion
`test_storage.py`	2	Création automatique des fichiers, sauvegarde et rechargement des utilisateurs
`test_quiz.py`	6	Création de quiz par un administrateur, refus pour un joueur, import depuis un fichier, quiz de démonstration
`test_import.py`	4	Lecture du format texte, barème `[1 pt]`, fichier introuvable, fichier non texte
`test_resultat.py`	5	Enregistrement des scores, refus des scores incohérents, historique propre à chaque joueur, statistiques

Des parcours complets ont également été testés manuellement en console et en interface graphique : connexion de l'administrateur, import d'un quiz, connexion d'un joueur, passation du quiz et consultation de l'historique.

E. Bilan final

E.1 Résultats obtenus

Toutes les exigences minimales du cahier des charges sont satisfaites :

Exigence	Résultat
Création de compte	Réalisé : inscription, connexion, rôles joueur et administrateur, mots de passe hachés
Historique des scores individuels	Réalisé : enregistrement automatique, historique avec moyenne et meilleur score
Création de quiz par l'administrateur, directement	Réalisé, en console et en interface graphique
Création de quiz depuis un fichier texte	Réalisé, en console et en interface graphique
Affichage du score final et des bonnes réponses	Réalisé
Interface graphique (optionnelle)	Réalisée
Minuterie, export JSON (optionnels)	Non réalisés

Côté gestion de projet, tous les livrables demandés sont présents dans le dépôt : WBS, PBS, Gantt, MCD, README, glossaire, comptes rendus des cinq séances et présent rapport.

E.2 Évaluation critique

- **Utilisation de Git :** l'équipe a principalement travaillé directement sur la branche `main`, sans branche par fonctionnalité ni Pull Request relue. Cela reste gérable pour quatre personnes, mais c'est à l'origine du conflit et du bug de la séance 4. Certains messages de commit (« Update cli.py ») ne décrivent pas le changement effectué.
- **Planning :** le Gantt prévoyait une fin le 06/11 ; une séance supplémentaire a été nécessaire pour finaliser le rapport et la soutenance. La fin du Sprint 4 était trop chargée (historique, corrections, interface graphique, rapport).
- **Tests :** la logique métier est bien couverte, mais l'interface graphique n'est testée que manuellement.
- **Sécurité :** SHA-256 avec sel est correct pour un projet pédagogique, mais un algorithme volontairement lent (PBKDF2, bcrypt) serait préférable en conditions réelles.
- **Points forts :** une architecture modulaire qui a permis d'ajouter l'interface graphique sans toucher à la logique métier, une application installable sans aucune dépendance, et une documentation complète.

E.3 Pistes d'amélioration et perspectives

- **Organisation :** une branche par user story et une Pull Request relue par un autre membre avant chaque fusion ; des messages de commit normalisés dès le début ; une marge dans le planning pour la rédaction du rapport.

- **Qualité :** exécution automatique des tests à chaque envoi sur GitHub (GitHub Actions) ; tests de l'interface graphique.
- **Fonctionnalités :** minuterie par question ou par quiz ; export et import de quiz au format JSON ; modification et suppression de quiz par l'administrateur ; statistiques globales par quiz pour l'administrateur.

E.4 Documentation réalisée

Document	Emplacement
README (objectif, installation, utilisation)	`README.md`
Comptes rendus des 5 séances	`docs/comptes-rendus/`
WBS, PBS, Gantt	`docs/wbs_projet_quiz_python.png`, `docs/PBS_Quizora.png`, `docs/Gantt_projet_quiz.gan`
MCD et dictionnaire des données	`docs/mcd_quizora.png`, `docs/mcd_documentation.md`
Dossier d'architecture technique	`docs/architecture_technique_quizora.pdf` (et versions Markdown et HTML)
Glossaire	`docs/glossaire.md`
Captures du tableau Trello	`docs/trello_sprint2.png` à `docs/trello_sprint5.png`
Rapport final	`docs/rapport_final.md` et `docs/rapport_final.pdf`

Conclusion

Le projet Quizora a abouti à une application complète qui répond à toutes les exigences minimales du sujet et y ajoute une interface graphique. Sur le plan de la gestion de projet, la démarche Scrum, le suivi sur Trello et les comptes rendus de chaque séance ont permis de livrer régulièrement des versions fonctionnelles et de garder une trace des décisions. Les difficultés rencontrées, notamment autour de l'utilisation de Git en équipe et du respect du planning, nous ont apporté des enseignements concrets que nous appliquerons dans nos prochains projets : travailler sur des branches avec relecture, automatiser les tests et prévoir une marge pour les livrables de fin de projet.